```
!pip install pillow matplotlib
!pip install tensorflow-gpu==2.0.0
```

```
Requirement already satisfied: pillow in
/usr/local/lib/python3.7/dist-packages (7.1.2)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.7/dist-packages (3.2.2)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.7/dist-packages (from matplotlib) (0.11.0)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!
=2.1.6,>=2.0.1 in /usr/local/lib/python3.7/dist-packages (from
matplotlib) (3.0.7)
Requirement already satisfied: python-dateutil>=2.1 in
/usr/local/lib/python3.7/dist-packages (from matplotlib) (2.8.2)
Requirement already satisfied: kiwisolver>=1.0.1 in
/usr/local/lib/python3.7/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: numpy>=1.11 in
/usr/local/lib/python3.7/dist-packages (from matplotlib) (1.21.5)
Requirement already satisfied: typing-extensions in
/usr/local/lib/python3.7/dist-packages (from kiwisolver>=1.0.1-
>matplotlib) (3.10.0.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.7/dist-packages (from python-dateutil>=2.1-
>matplotlib) (1.15.0)
Requirement already satisfied: tensorflow-gpu==2.0.0 in
/usr/local/lib/python3.7/dist-packages (2.0.0)
Requirement already satisfied: google-pasta>=0.1.6 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(0.2.0)
Requirement already satisfied: numpy<2.0,>=1.16.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.21.5)
Requirement already satisfied: gast==0.2.2 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(0.2.2)
Requirement already satisfied: astor>=0.6.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(0.8.1)
Requirement already satisfied: keras-preprocessing>=1.0.5 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.1.2)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.1.0)
Requirement already satisfied: protobuf>=3.6.1 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(3.17.3)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(3.3.0)
```

```
Requirement already satisfied: absl-py>=0.7.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.0.0)
Requirement already satisfied: tensorflow-estimator<2.1.0,>=2.0.0
in /usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(2.0.1)
Requirement already satisfied: grpcio>=1.8.6 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.44.0)
Requirement already satisfied: six>=1.10.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.15.0)
Requirement already satisfied: wheel>=0.26 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(0.37.1)
Requirement already satisfied: keras-applications>=1.0.8 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.0.8)
Requirement already satisfied: tensorboard<2.1.0,>=2.0.0 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(2.0.2)
Requirement already satisfied: wrapt>=1.11.1 in
/usr/local/lib/python3.7/dist-packages (from tensorflow-gpu==2.0.0)
(1.14.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.7/dist-
packages (from keras-applications>=1.0.8->tensorflow-gpu==2.0.0)
(3.1.0)
Requirement already satisfied: google-auth-oauthlib<0.5,>=0.4.1 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (0.4.6)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (3.3.6)
Requirement already satisfied: setuptools>=41.0.0 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (57.4.0)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (2.23.0)
Requirement already satisfied: google-auth<2,>=1.6.3 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (1.35.0)
Requirement already satisfied: werkzeug>=0.11.15 in
/usr/local/lib/python3.7/dist-packages (from
tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (1.0.1)
Requirement already satisfied: rsa<5,>=3.1.4 in
/usr/local/lib/python3.7/dist-packages (from google-auth<2,>=1.6.3-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (4.8)
Requirement already satisfied: cachetools<5.0,>=2.0.0 in
/usr/local/lib/python3.7/dist-packages (from google-auth<2,>=1.6.3-
```

```
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (4.2.4)
Requirement already satisfied: pyasn1-modules>=0.2.1 in
/usr/local/lib/python3.7/dist-packages (from google-auth<2,>=1.6.3-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (0.2.8)
Requirement already satisfied: requests-oauthlib>=0.7.0 in
/usr/local/lib/python3.7/dist-packages (from google-auth-
oauthlib<0.5,>=0.4.1->tensorboard<2.1.0,>=2.0.0->tensorflow-
gpu==2.0.0) (1.3.1)
Requirement already satisfied: importlib-metadata>=4.4 in
/usr/local/lib/python3.7/dist-packages (from markdown>=2.6.8-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (4.11.3)
Requirement already satisfied: zipp>=0.5 in
/usr/local/lib/python3.7/dist-packages (from importlib-metadata>=4.4-
>markdown>=2.6.8->tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0)
(3.7.0)
Requirement already satisfied: typing-extensions>=3.6.4 in
/usr/local/lib/python3.7/dist-packages (from importlib-metadata>=4.4-
>markdown>=2.6.8->tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0)
(3.10.0.2)
Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in
/usr/local/lib/python3.7/dist-packages (from pyasn1-modules>=0.2.1-
>google-auth<2,>=1.6.3->tensorboard<2.1.0,>=2.0.0->tensorflow-
gpu==2.0.0) (0.4.8)
Requirement already satisfied: idna<3,>=2.5 in
/usr/local/lib/python3.7/dist-packages (from requests<3,>=2.21.0-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (2.10)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.7/dist-packages (from requests<3,>=2.21.0-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (2021.10.8)
Requirement already satisfied: chardet<4,>=3.0.2 in
/usr/local/lib/python3.7/dist-packages (from requests<3,>=2.21.0-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (3.0.4)
Requirement already satisfied: urllib3!=1.25.0,!=1.25.1,<1.26,>=1.21.1
in /usr/local/lib/python3.7/dist-packages (from requests<3,>=2.21.0-
>tensorboard<2.1.0,>=2.0.0->tensorflow-gpu==2.0.0) (1.24.3)
Requirement already satisfied: oauthlib>=3.0.0 in
/usr/local/lib/python3.7/dist-packages (from requests-oauthlib>=0.7.0-
>google-auth-oauthlib<0.5,>=0.4.1->tensorboard<2.1.0,>=2.0.0-
>tensorflow-gpu==2.0.0) (3.2.0)
Requirement already satisfied: cached-property in
/usr/local/lib/python3.7/dist-packages (from h5py->keras-
applications>=1.0.8->tensorflow-gpu==2.0.0) (1.5.2)
```

```
!pip install 'h5py==2.10.0'
```

```
Collecting h5py==2.10.0
  Downloading h5py-2.10.0-cp37-cp37m-manylinux1_x86_64.whl (2.9 MB)
ent already satisfied: six in /usr/local/lib/python3.7/dist-packages
(from h5py==2.10.0) (1.15.0)
Requirement already satisfied: numpy>=1.7 in
/usr/local/lib/python3.7/dist-packages (from h5py==2.10.0) (1.21.5)
```

```
Installing collected packages: h5py
  Attempting uninstall: h5py
    Found existing installation: h5py 3.1.0
    Uninstalling h5py-3.1.0:
      Successfully uninstalled h5py-3.1.0
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
tensorflow 2.8.0 requires tf-estimator-nightly==2.8.0.dev2021122109,
which is not installed.
tensorflow 2.8.0 requires tensorboard<2.9,>=2.8, but you have
tensorboard 2.0.2 which is incompatible.
Successfully installed h5py-2.10.0
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
# Copyright 2019 NVIDIA Corporation. All Rights Reserved.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
==========================================================================
========
```

TF-TRT Inference from Keras Model with TensorFlow 2.0

Introduction

The NVIDIA TensorRT is a C++ library that facilitates high performance inference on NVIDIA graphics processing units (GPUs). TensorRT takes a trained network, which consists of a network definition and a set of trained parameters, and produces a highly optimized runtime engine which performs inference for that network.

TensorFlow™ integration with TensorRT™ (TF-TRT) optimizes and executes compatible subgraphs, allowing TensorFlow to execute the remaining graph. While you can still use TensorFlow's wide and flexible feature set, TensorRT will parse the model and apply optimizations to the portions of the graph wherever possible.

In this notebook, we demonstrate the process of creating a TF-TRT optimized model from a VGG-16 Keras saved model.

Requirement

GPU

Before running this notebook, please set the Colab runtime environment to GPU via the menu *Runtime => Change runtime type => GPU*.

This demo will work on any NVIDIA GPU with CUDA cores, though for improved FP16 and INT8 inference, a Volta, Turing or newer generation GPU with Tensor cores is desired. On Google Colab, this normally means a T4 GPU. If you are assigned an older K80 GPU, another trial at another time might give you a T4 GPU.

```
!nvidia-smi
```

```
Sat Apr  9 06:25:28 2022
+-----------------------------------------------------------------------------
--------+
| NVIDIA-SMI 460.32.03    Driver Version: 460.32.03    CUDA Version:
11.2     |
|-------------------------------+----------------------
+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util
Compute M. |
|                               |                      |
MIG M. |
|
===============================+======================+===============
=======|
|   0  Tesla K80           Off  | 00000000:00:04.0 Off |
0 |
| N/A   64C    P8    32W / 149W |      0MiB / 11441MiB |      0%
Default |
|                               |                      |
N/A |
+-------------------------------+----------------------
+----------------------+


+-----------------------------------------------------------------------------
--------+
```

```
| Processes:
|
|  GPU   GI   CI        PID   Type   Process name                  GPU
Memory |
|        ID   ID
Usage      |
|
=============================================================================
=======|
|  No running processes found
|
+-----------------------------------------------------------------------------
--------+
```

Install TensorFlow-GPU 2.0

```
import tensorflow as tf
print("Tensorflow version: ", tf.version.VERSION)
```

```
Tensorflow version:  2.0.0
```

Install TensorRT Runtime

```
%%bash
wget https://developer.download.nvidia.com/compute/machine-
learning/repos/ubuntu1804/x86_64/nvidia-machine-learning-repo-
ubuntu1804_1.0.0-1_amd64.deb

dpkg -i nvidia-machine-learning-repo-*.deb
apt-get update

sudo apt-get install libnvinfer5
```

```
Selecting previously unselected package nvidia-machine-learning-repo-
ubuntu1804.
(Reading database ... 156210 files and directories currently
installed.)
Preparing to unpack nvidia-machine-learning-repo-ubuntu1804_1.0.0-
1_amd64.deb ...
Unpacking nvidia-machine-learning-repo-ubuntu1804 (1.0.0-1) ...
Setting up nvidia-machine-learning-repo-ubuntu1804 (1.0.0-1) ...
Get:1 https://cloud.r-project.org/bin/linux/ubuntu bionic-cran40/
InRelease [3,626 B]
Ign:2
http://developer.download.nvidia.com/compute/machine-learning/repos/
ubuntu1804/x86_64  InRelease
Hit:3
http://developer.download.nvidia.com/compute/machine-learning/repos/
ubuntu1804/x86_64  Release
Ign:4
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/
x86_64  InRelease
Get:5
```

```
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/
x86_64  Release [696 B]
Get:6
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/
x86_64  Release.gpg [836 B]
Get:7 http://security.ubuntu.com/ubuntu bionic-security InRelease
[88.7 kB]
Get:8 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu bionic
InRelease [15.9 kB]
Hit:9 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:10 https://cloud.r-project.org/bin/linux/ubuntu bionic-cran40/
Packages [82.3 kB]
Get:11 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7
kB]
Hit:13 http://ppa.launchpad.net/cran/libgit2/ubuntu bionic InRelease
Get:14 http://archive.ubuntu.com/ubuntu bionic-backports InRelease
[74.6 kB]
Get:15
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/
x86_64  Packages [952 kB]
Get:16 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu bionic InRelease
[15.9 kB]
Get:17 http://ppa.launchpad.net/graphics-drivers/ppa/ubuntu bionic
InRelease [21.3 kB]
Get:18 http://security.ubuntu.com/ubuntu bionic-security/universe
amd64 Packages [1,490 kB]
Get:19 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu
bionic/main Sources [1,831 kB]
Get:20 http://archive.ubuntu.com/ubuntu bionic-updates/universe amd64
Packages [2,268 kB]
Get:21 http://security.ubuntu.com/ubuntu bionic-security/restricted
amd64 Packages [884 kB]
Get:22 http://security.ubuntu.com/ubuntu bionic-security/main amd64
Packages [2,694 kB]
Get:23 http://archive.ubuntu.com/ubuntu bionic-updates/multiverse
amd64 Packages [29.8 kB]
Get:24 http://archive.ubuntu.com/ubuntu bionic-updates/restricted
amd64 Packages [918 kB]
Get:25 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64
Packages [3,133 kB]
Get:26 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu
bionic/main amd64 Packages [938 kB]
Get:27 http://archive.ubuntu.com/ubuntu bionic-backports/main amd64
Packages [12.2 kB]
Get:28 http://archive.ubuntu.com/ubuntu bionic-backports/universe
amd64 Packages [12.9 kB]
Get:29 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu bionic/main
amd64 Packages [45.2 kB]
Get:30 http://ppa.launchpad.net/graphics-drivers/ppa/ubuntu
bionic/main amd64 Packages [44.3 kB]
```

```
Fetched 15.6 MB in 4s (3,941 kB/s)
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...
The following NEW packages will be installed:
  libnvinfer5
0 upgraded, 1 newly installed, 0 to remove and 96 not upgraded.
Need to get 46.7 MB of archives.
After this operation, 163 MB of additional disk space will be used.
Get:1
http://developer.download.nvidia.com/compute/machine-learning/repos/
ubuntu1804/x86_64  libnvinfer5 5.1.5-1+cuda10.1 [46.7 MB]
Fetched 46.7 MB in 1s (53.6 MB/s)
Selecting previously unselected package libnvinfer5.
(Reading database ...  (Reading database ... 5% (Reading database ...
10% (Reading database ... 15% (Reading database ... 20% (Reading
database ... 25% (Reading database ... 30% (Reading database ... 35%
(Reading database ... 40% (Reading database ... 45% (Reading
database ... 50% (Reading database ... 55% (Reading database ... 60%
(Reading database ... 65% (Reading database ... 70% (Reading
database ... 75% (Reading database ... 80% (Reading database ... 85%
(Reading database ... 90% (Reading database ... 95% (Reading
database ... 100% (Reading database ... 156213 files and directories
currently installed.)
Preparing to unpack .../libnvinfer5_5.1.5-1+cuda10.1_amd64.deb ...
Unpacking libnvinfer5 (5.1.5-1+cuda10.1) ...
Setting up libnvinfer5 (5.1.5-1+cuda10.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1.3) ...
/sbin/ldconfig.real:
/usr/local/lib/python3.7/dist-packages/ideep4py/lib/libmkldnn.so.0 is
not a symbolic link


--2022-04-09 06:25:47--
https://developer.download.nvidia.com/compute/machine-learning/repos/
ubuntu1804/x86_64/nvidia-machine-learning-repo-ubuntu1804_1.0.0-
1_amd64.deb
Resolving developer.download.nvidia.com
(developer.download.nvidia.com)... 152.195.19.142
Connecting to developer.download.nvidia.com
(developer.download.nvidia.com)|152.195.19.142|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2926 (2.9K) [application/x-deb]
Saving to: ‘nvidia-machine-learning-repo-ubuntu1804_1.0.0-1_amd64.deb’

     0K ..                                                    100%
109M=0s

2022-04-09 06:25:47 (109 MB/s) - ‘nvidia-machine-learning-repo-
```

```
ubuntu1804_1.0.0-1_amd64.deb’ saved [2926/2926]

W: Target Packages (Packages) is configured multiple times in
/etc/apt/sources.list.d/nvidia-machine-learning.list:1 and
/etc/apt/sources.list.d/nvidia-ml.list:1
W: Target Packages (Packages) is configured multiple times in
/etc/apt/sources.list.d/nvidia-machine-learning.list:1 and
/etc/apt/sources.list.d/nvidia-ml.list:1
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog
based frontend cannot be used. at
/usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 76, <> line 1.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
W: Target Packages (Packages) is configured multiple times in
/etc/apt/sources.list.d/nvidia-machine-learning.list:1 and
/etc/apt/sources.list.d/nvidia-ml.list:1
```

```
# check TensorRT version
print("TensorRT version: ")
!dpkg -l | grep nvinfer
```

```
TensorRT version:
ii  libnvinfer5                              5.1.5-1+cuda10.1
amd64        TensorRT runtime libraries
```

A successfull TensorRT installation looks like:

```
TensorRT version:
ii  libnvinfer5   5.1.5-1+cuda10.1   amd64        TensorRT runtime
libraries
```

Check Tensor core GPU

The below code check whether a Tensor-core GPU is present.

```
from tensorflow.python.client import device_lib

def check_tensor_core_gpu_present():
    local_device_protos = device_lib.list_local_devices()
    for line in local_device_protos:
        if "compute capability" in str(line):
            compute_capability =
float(line.physical_device_desc.split("compute capability: ")[-1])
            if compute_capability>=7.0:
                return True
```

```
print("Tensor Core GPU Present:", check_tensor_core_gpu_present())
tensor_core_gpu = check_tensor_core_gpu_present()
```

```
Tensor Core GPU Present: None
```

Importing required libraries

```
from __future__ import absolute_import, division, print_function,
unicode_literals
import os
import time

import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.python.compiler.tensorrt import trt_convert as trt
from tensorflow.python.saved_model import tag_constants
from tensorflow.keras.applications.resnet50 import ResNet50
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import preprocess_input,
decode_predictions
from tensorflow.keras.applications.vgg16 import
VGG16,preprocess_input, decode_predictions

from tensorflow.keras.applications.inception_v3 import
InceptionV3,preprocess_input, decode_predictions
```

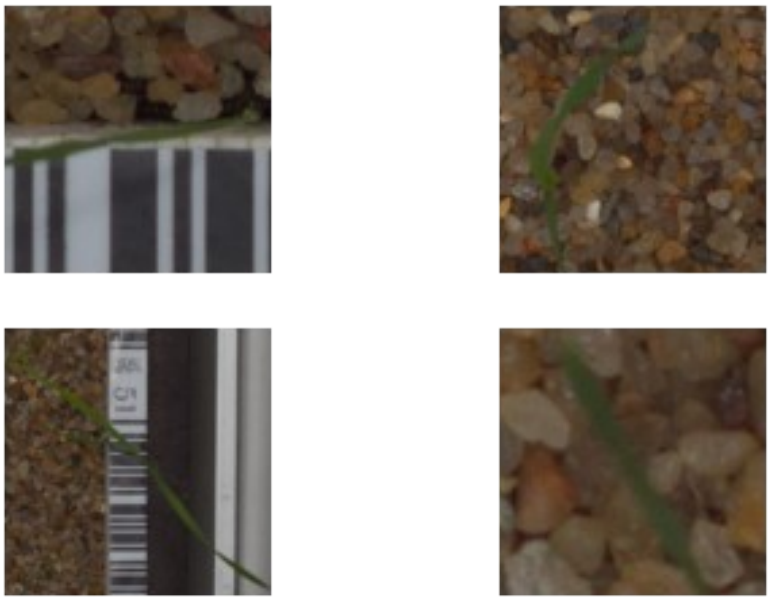
```
image_dir = '/content/drive/MyDrive/Colab Notebooks/test'
images = os.listdir(image_dir)
images[1]
```

```
{"type":"string"}
```

```
from tensorflow.keras.preprocessing import image

fig, axes = plt.subplots(nrows=2, ncols=2)

for i in range(4):
  img_file = images[i]
  img_path = os.path.join(image_dir,img_file)
  img = image.load_img(img_path, target_size=(224, 224))
  plt.subplot(2,2,i+1)
  plt.imshow(img);
  plt.axis('off');
```



Model

We next download and test a VGG16 pre-trained model from the Keras model zoo.

```
# model = ResNet50(weights='imagenet')
#model = InceptionV3(weights = 'imagenet')
model = VGG16(weights= 'imagenet')
```

```
Downloading data from https://github.com/fchollet/deep-learning-
models/releases/download/v0.1/
vgg16_weights_tf_dim_ordering_tf_kernels.h5
553467904/553467096 [==============================] - 5s 0us/step
```

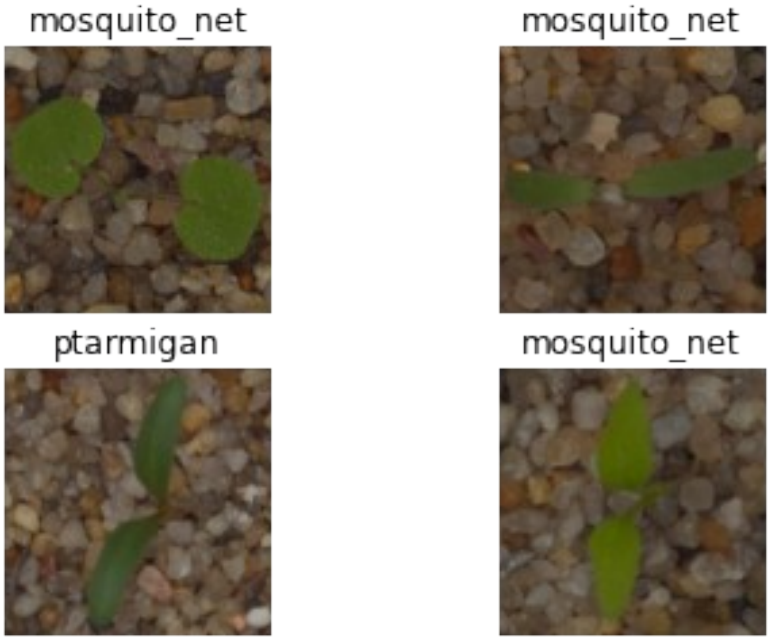
```
for i in range(4):
  img_file = images[i]
  img_path = os.path.join(image_dir,img_file)
  img = image.load_img(img_path, target_size=(224, 224))
  x = image.img_to_array(img)
  x = np.expand_dims(x, axis=0)
  x = preprocess_input(x)

  preds = model.predict(x)
  # decode the results into a list of tuples (class, description,
probability)
  # (one such list for each sample in the batch)
  print('{} - Predicted: {}'.format(img_path,
decode_predictions(preds, top=3)[0]))

  plt.subplot(2,2,i+1)
  plt.imshow(img);
  plt.axis('off');
```

```
  plt.title(decode_predictions(preds, top=3)[0][0][1])
```

```
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/imagenet_c
lass_index.json
40960/35363 [==================================] - 0s 0us/step
/content/drive/MyDrive/Colab Notebooks/test/0021e90e4.png - Predicted:
[('n03788365', 'mosquito_net', 0.04051131), ('n07860988', 'dough',
0.016027138), ('n01498041', 'stingray', 0.014224683)]
/content/drive/MyDrive/Colab Notebooks/test/003d61042.png - Predicted:
[('n03788365', 'mosquito_net', 0.037750714), ('n15075141',
'toilet_tissue', 0.014821087), ('n07860988', 'dough', 0.014235996)]
/content/drive/MyDrive/Colab Notebooks/test/007b3da8b.png - Predicted:
[('n01796340', 'ptarmigan', 0.015751045), ('n03788365',
'mosquito_net', 0.014894152), ('n07860988', 'dough', 0.0144516155)]
/content/drive/MyDrive/Colab Notebooks/test/0086a6340.png - Predicted:
[('n03788365', 'mosquito_net', 0.021375598), ('n01796340',
'ptarmigan', 0.016553488), ('n07860988', 'dough', 0.012831482)]
```



TF-TRT takes input as aTensorFlow saved model, therefore, we re-export the Keras model as a TF saved model.

```
# Save the entire model as a SavedModel.
model.save('/content/drive/MyDrive/Colab Notebooks/vgg16_saved_model')
```

```
WARNING:tensorflow:From
/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/ops/
resource_variable_ops.py:1781: calling BaseResourceVariable.__init__
(from tensorflow.python.ops.resource_variable_ops) with constraint is
deprecated and will be removed in a future version.
```

```
Instructions for updating:
If using Keras pass *_constraint arguments to layers.
INFO:tensorflow:Assets written to: /content/drive/MyDrive/Colab
Notebooks/vgg16_saved_model/assets
```

```
!saved_model_cli show --all --dir vgg16_saved_model
```

```
Traceback (most recent call last):
  File "/usr/local/bin/saved_model_cli", line 8, in <module>
    sys.exit(main())
  File
"/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/tools/
saved_model_cli.py", line 909, in main
    args.func(args)
  File
"/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/tools/
saved_model_cli.py", line 610, in show
    _show_all(args.dir)
  File
"/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/tools/
saved_model_cli.py", line 192, in _show_all
    tag_sets =
saved_model_utils.get_saved_model_tag_sets(saved_model_dir)
  File
"/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/tools/
saved_model_utils.py", line 88, in get_saved_model_tag_sets
    saved_model = read_saved_model(saved_model_dir)
  File
"/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/tools/
saved_model_utils.py", line 55, in read_saved_model
    raise IOError("SavedModel file does not exist at: %s" %
saved_model_dir)
OSError: SavedModel file does not exist at: vgg16_saved_model
```

Inference with native TF2.0 saved model

```
model = tf.keras.models.load_model('/content/drive/MyDrive/Colab
Notebooks/vgg16_saved_model')
```

```
img_file = images[i]
img_path = os.path.join(image_dir,img_file)
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

preds = model.predict(x)
# decode the results into a list of tuples (class, description,
probability)
# (one such list for each sample in the batch)
print('{} - Predicted: {}'.format(img_path, decode_predictions(preds,
top=3)[0]))
```

```
plt.subplot(2,2,1)
plt.imshow(img);
plt.axis('off');
plt.title(decode_predictions(preds, top=3)[0][0][1])
```

```
/content/drive/MyDrive/Colab Notebooks/test/0086a6340.png - Predicted:
[('n03788365', 'mosquito_net', 0.021375598), ('n01796340',
'ptarmigan', 0.016553488), ('n07860988', 'dough', 0.012831482)]
```

```
Text(0.5, 1.0, 'mosquito_net')
```




```
batch_size = 8
batched_input = np.zeros((batch_size, 224, 224, 3), dtype=np.float32)

for i in range(batch_size):
  img_file = images[i%4]
  img_path = os.path.join(image_dir,img_file)
  img = image.load_img(img_path, target_size=(224, 224))
  x = image.img_to_array(img)
  x = np.expand_dims(x, axis=0)
  x = preprocess_input(x)
  batched_input[i, :] = x
batched_input = tf.constant(batched_input)
print('batched_input shape: ', batched_input.shape)
```

```
batched_input shape:  (8, 224, 224, 3)
```

```
# Benchmarking throughput

N_run = 300
elapsed_time = []

for i in range(N_run):
  start_time = time.time()
  preds = model.predict(batched_input)
  end_time = time.time()
  elapsed_time = np.append(elapsed_time, end_time - start_time)
  if i % 50 == 0:
    print('Step {}: {:4.1f}ms'.format(i, (elapsed_time[-50:].mean()) *
1000))
```

```
print('Throughput: {:.0f} images/s'.format(N_run * batch_size /
elapsed_time.sum()))
```

```
Step 0: 1032.0ms
Step 50: 52.6ms
Step 100: 54.2ms
Step 150: 54.1ms
Step 200: 52.6ms
Step 250: 52.6ms
Throughput: 142 images/s
```

TF-TRT FP32 model

We first convert the TF native FP32 model to a TF-TRT FP32 model.

```
print('Converting to TF-TRT FP32...')
conversion_params =
trt.DEFAULT_TRT_CONVERSION_PARAMS._replace(precision_mode=trt.TrtPreci
sionMode.FP32,

max_workspace_size_bytes=8000000000)

converter =
trt.TrtGraphConverterV2(input_saved_model_dir='/content/drive/MyDrive/
Colab Notebooks/vgg16_saved_model',

conversion_params=conversion_params)
converter.convert()
converter.save(output_saved_model_dir='vgg16_saved_model_TFTRT_FP32')
print('Done Converting to TF-TRT FP32')
```

```
Converting to TF-TRT FP32...
INFO:tensorflow:Linked TensorRT version: (5, 1, 5)
INFO:tensorflow:Loaded TensorRT version: (5, 1, 5)
INFO:tensorflow:Running against TensorRT version 5.1.5
WARNING:tensorflow:From
/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/ops/
resource_variable_ops.py:1781: calling BaseResourceVariable.__init__
(from tensorflow.python.ops.resource_variable_ops) with constraint is
deprecated and will be removed in a future version.
Instructions for updating:
If using Keras pass *_constraint arguments to layers.
INFO:tensorflow:Assets written to: vgg16_saved_model_TFTRT_FP32/assets
Done Converting to TF-TRT FP32
```

```
!saved_model_cli show --all --dir vgg16_saved_model_TFTRT_FP32
```

```
MetaGraphDef with tag-set: 'serve' contains the following
SignatureDefs:
```

```
signature_def['__saved_model_init_op']:
  The given SavedModel SignatureDef contains the following input(s):
  The given SavedModel SignatureDef contains the following output(s):
    outputs['__saved_model_init_op'] tensor_info:
        dtype: DT_INVALID
        shape: unknown_rank
        name: NoOp
  Method name is:

signature_def['serving_default']:
  The given SavedModel SignatureDef contains the following input(s):
    inputs['input_1'] tensor_info:
        dtype: DT_FLOAT
        shape: (-1, 224, 224, 3)
        name: serving_default_input_1:0
  The given SavedModel SignatureDef contains the following output(s):
    outputs['predictions'] tensor_info:
        dtype: DT_FLOAT
        shape: unknown_rank
        name: PartitionedCall:0
  Method name is: tensorflow/serving/predict
```

Next, we load and test the TF-TRT FP32 model.

```
def predict_tftrt(input_saved_model):
    """Runs prediction on a single image and shows the result.
    input_saved_model (string): Name of the input model stored in the
current dir
    """
    img_file = images[0]
    img_path = os.path.join(image_dir,img_file)
    img = image.load_img(img_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = preprocess_input(x)
    x = tf.constant(x)

    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    signature_keys = list(saved_model_loaded.signatures.keys())
    print(signature_keys)

    infer = saved_model_loaded.signatures['serving_default']
    print(infer.structured_outputs)

    labeling = infer(x)
    print(labeling.keys())
    #preds = labeling['probs'].numpy()
    preds = labeling['predictions'].numpy()
```

```
    print('{} - Predicted: {}'.format(img_path,
decode_predictions(preds, top=3)[0]))
    plt.subplot(2,2,1)
    plt.imshow(img);
    plt.axis('off');
    plt.title(decode_predictions(preds, top=3)[0][0][1])

predict_tftrt('vgg16_saved_model_TFTRT_FP32')
```

```
['serving_default']
{'predictions': TensorSpec(shape=<unknown>, dtype=tf.float32,
name='predictions')}
dict_keys(['predictions'])
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/imagenet_c
lass_index.json
40960/35363 [==================================] - 0s 0us/step
/content/drive/MyDrive/Colab Notebooks/train/Black-grass/0050f38b3.png
- Predicted: [('n03788365', 'mosquito_net', 0.07154264), ('n15075141',
'toilet_tissue', 0.031133646), ('n04209239', 'shower_curtain',
0.022633843)]
```




```
def benchmark_tftrt(input_saved_model):
    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    infer = saved_model_loaded.signatures['serving_default']


    N_run = 300
    elapsed_time = []

    for i in range(N_run):
      start_time = time.time()
      labeling = infer(batched_input)
      #prob = labeling['probs'].numpy()
      end_time = time.time()
      elapsed_time = np.append(elapsed_time, end_time - start_time)
      if i % 50 == 0:
        print('Step {}: {:4.1f}ms'.format(i, (elapsed_time[-
50:].mean()) * 1000))
```

```
    print('Throughput: {:.0f} images/s'.format(N_run * batch_size /
elapsed_time.sum()))

benchmark_tftrt('vgg16_saved_model_TFTRT_FP32')
```

```
Step 0: 13852.6ms
Step 50: 47.1ms
Step 100: 48.3ms
Step 150: 48.6ms
Step 200: 48.6ms
Step 250: 48.7ms
Throughput: 85 images/s
```

TF-TRT FP16 model

We next convert the native TF FP32 saved model to TF-TRT FP16 model.

```
def predict_tftrt(input_saved_model):
    """Runs prediction on a single image and shows the result.
    input_saved_model (string): Name of the input model stored in the
current dir
    """
    img_file = images[0]
    img_path = os.path.join(image_dir,img_file)
    img = image.load_img(img_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = preprocess_input(x)
    x = tf.constant(x)

    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    signature_keys = list(saved_model_loaded.signatures.keys())
    print(signature_keys)

    infer = saved_model_loaded.signatures['serving_default']
    print(infer.structured_outputs)

    labeling = infer(x)
    preds = labeling['predictions'].numpy()
    print('{} - Predicted: {}'.format(img_path,
decode_predictions(preds, top=3)[0]))
    plt.subplot(2,2,1)
    plt.imshow(img);
    plt.axis('off');
    plt.title(decode_predictions(preds, top=3)[0][0][1])

print('Converting to TF-TRT FP16...')
conversion_params = trt.DEFAULT_TRT_CONVERSION_PARAMS._replace(
    precision_mode=trt.TrtPrecisionMode.FP16,
```

```
    max_workspace_size_bytes=8000000000)
converter = trt.TrtGraphConverterV2(
   input_saved_model_dir='/content/drive/MyDrive/Colab
Notebooks/vgg16_saved_model', conversion_params=conversion_params)
converter.convert()
converter.save(output_saved_model_dir='vgg16_saved_model_TFTRT_FP16')
print('Done Converting to TF-TRT FP16')
```

```
Converting to TF-TRT FP16...
INFO:tensorflow:Linked TensorRT version: (5, 1, 5)
INFO:tensorflow:Loaded TensorRT version: (5, 1, 5)
INFO:tensorflow:Running against TensorRT version 5.1.5
WARNING:tensorflow:From
/usr/local/lib/python3.7/dist-packages/tensorflow_core/python/ops/
resource_variable_ops.py:1781: calling BaseResourceVariable.__init__
(from tensorflow.python.ops.resource_variable_ops) with constraint is
deprecated and will be removed in a future version.
Instructions for updating:
If using Keras pass *_constraint arguments to layers.
INFO:tensorflow:Assets written to: vgg16_saved_model_TFTRT_FP16/assets
Done Converting to TF-TRT FP16
```

```
predict_tftrt('vgg16_saved_model_TFTRT_FP16')
```

```
['serving_default']
{'predictions': TensorSpec(shape=<unknown>, dtype=tf.float32,
name='predictions')}
/content/drive/MyDrive/Colab Notebooks/train/Black-grass/0050f38b3.png
- Predicted: [('n03788365', 'mosquito_net', 0.07154888), ('n15075141',
'toilet_tissue', 0.03112983), ('n04209239', 'shower_curtain',
0.022629205)]
```




```
def benchmark_tftrt(input_saved_model):
    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    infer = saved_model_loaded.signatures['serving_default']


    N_run = 300
    elapsed_time = []
```

```
    for i in range(N_run):
      start_time = time.time()
      labeling = infer(batched_input)
      #prob = labeling['probs'].numpy()
      end_time = time.time()
      elapsed_time = np.append(elapsed_time, end_time - start_time)
      if i % 50 == 0:
        print('Step {}: {:4.1f}ms'.format(i, (elapsed_time[-
50:].mean()) * 1000))

    print('Throughput: {:.0f} images/s'.format(N_run * batch_size /
elapsed_time.sum()))

benchmark_tftrt('vgg16_saved_model_TFTRT_FP16')
```

```
Step 0: 4364.5ms
Step 50: 49.7ms
Step 100: 48.9ms
Step 150: 49.0ms
Step 200: 49.2ms
Step 250: 49.3ms
Throughput: 126 images/s
```

TF-TRT INT8 model

Creating TF-TRT INT8 model requires a small calibration dataset. This data set ideally should represent the test data in production well, and will be used to create a value histogram for each layer in the neural network for effective 8-bit quantization.

Herein, for demonstration purposes, we take only the 4 images that we downloaded for calibration. In production, this set should be more representative of the production data.

Due to Colab memory limit which sometime causes TensorRT to crash, to proceed, first, restart the runtime by pressing CTRL+M or select *Runtime -> Restart runtime...* or simply execute the next cell.

```
# import os
# os.kill(os.getpid(), 9)
```

```
batch_size = 8
batched_input = np.zeros((batch_size, 224, 224, 3), dtype=np.float32)

for i in range(batch_size):
  img_file = images[i%4]
  img_path = os.path.join(image_dir,img_file)
  img = image.load_img(img_path, target_size=(224, 224))
  x = image.img_to_array(img)
  x = np.expand_dims(x, axis=0)
  x = preprocess_input(x)
  batched_input[i, :] = x
```

```
batched_input = tf.constant(batched_input)
print('batched_input shape: ', batched_input.shape)
```

```
batched_input shape:  (8, 224, 224, 3)
```

```
print('Converting to TF-TRT INT8...')
conversion_params = trt.DEFAULT_TRT_CONVERSION_PARAMS._replace(
    precision_mode=trt.TrtPrecisionMode.INT8,
    max_workspace_size_bytes=8000000000,
    use_calibration=True)
converter = trt.TrtGraphConverterV2(
    input_saved_model_dir='/content/drive/MyDrive/Colab
Notebooks/vgg16_saved_model',
    conversion_params=conversion_params)

def calibration_input_fn():
    yield (batched_input, )
converter.convert(calibration_input_fn=calibration_input_fn)

converter.save(output_saved_model_dir='vgg16_saved_model_TFTRT_INT8')
print('Done Converting to TF-TRT INT8')
```

```
Converting to TF-TRT INT8...
INFO:tensorflow:Linked TensorRT version: (5, 1, 5)
INFO:tensorflow:Loaded TensorRT version: (5, 1, 5)
INFO:tensorflow:Running against TensorRT version 5.1.5
```

```
def predict_tftrt(input_saved_model):
    """Runs prediction on a single image and shows the result.
    input_saved_model (string): Name of the input model stored in the
current dir
    """
    img_file = images[0]
    img_path = os.path.join(image_dir,img_file)
    img = image.load_img(img_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = preprocess_input(x)
    x = tf.constant(x)

    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    signature_keys = list(saved_model_loaded.signatures.keys())
    print(signature_keys)

    infer = saved_model_loaded.signatures['serving_default']
    print(infer.structured_outputs)

    labeling = infer(x)
    preds = labeling['probs'].numpy()
    print('{} - Predicted: {}'.format(img_path,
```

```
decode_predictions(preds, top=3)[0]))
    plt.subplot(2,2,1)
    plt.imshow(img);
    plt.axis('off');
    plt.title(decode_predictions(preds, top=3)[0][0][1])

predict_tftrt('vgg16_saved_model_TFTRT_INT8')
```

```
['serving_default']
{'probs': TensorSpec(shape=<unknown>, dtype=tf.float32, name='probs')}
/content/drive/MyDrive/Colab Notebooks/train/Black-grass/0050f38b3.png
- Predicted: [('n03930313', 'picket_fence', 0.21861497), ('n03089624',
'confectionery', 0.21403234), ('n07880968', 'burrito', 0.098421164)]
```




```
def benchmark_tftrt(input_saved_model):
    saved_model_loaded = tf.saved_model.load(input_saved_model,
tags=[tag_constants.SERVING])
    infer = saved_model_loaded.signatures['serving_default']

    N_run = 300
    elapsed_time = []

    for i in range(N_run):
      start_time = time.time()
      labeling = infer(batched_input)
      #prob = labeling['probs'].numpy()
      end_time = time.time()
      elapsed_time = np.append(elapsed_time, end_time - start_time)
      if i % 50 == 0:
        print('Step {}: {:4.1f}ms'.format(i, (elapsed_time[-
50:].mean()) * 1000))

    print('Throughput: {:.0f} images/s'.format(N_run * batch_size /
elapsed_time.sum()))

benchmark_tftrt('vgg16_saved_model_TFTRT_INT8')
```

```
WARNING:tensorflow:Unresolved object in checkpoint:
(root).trt_engine_resources.TRTEngineOp_0._serialized_trt_resource_fil
ename
```

```
WARNING:tensorflow:A checkpoint was restored (e.g.
tf.train.Checkpoint.restore or tf.keras.Model.load_weights) but not
all checkpointed values were used. See above for specific issues. Use
expect_partial() on the load status object, e.g.
tf.train.Checkpoint.restore(...).expect_partial(), to silence these
warnings, or use assert_consumed() to make the check explicit. See
https://www.tensorflow.org/alpha/guide/checkpoints#loading_mechanics
for details.
Step 0: 1442.1ms
Step 50:  5.8ms
Step 100:  3.4ms
Step 150:  3.4ms
Step 200:  3.3ms
Step 250:  3.3ms
Throughput: 934 images/s
```

Conclusion

In this notebook, we have demonstrated the process of creating TF-TRT FP32, FP16 and INT8 inference models from an original Keras FP32 model, as well as verify their speed and accuracy.

What's next

Try TF-TRT on your own model and data, and experience the simplicity and speed up it offers.